Phase 3 user flow: runtime foundation

Delivered 4 October 2026 for the local Java prototype. Use this alongside the main user manual. Phase 4 has not started.

What this phase enables

ArchIntel receives real OpenTelemetry Java spans, associates them with a selected static analysis, and shows observed classes, methods, entry points, dependencies and domains. It saves the instrumentation scope and reports incomplete or conflicting telemetry. Static analysis results remain unchanged.

You do not have to connect an application to inspect a plan or create a session. Receiving traces requires starting an instrumented Java application and exercising its workflows. Creating a session does not launch or modify that application.

Connect and observe

1. Start ArchIntel with `Start-ArchIntel.cmd`, open http://127.0.0.1:8000 and sign in. Your account needs runtime read/write permission and access to the application's completed static analysis.
2. Open **Dynamic analysis**. Choose the application and the static baseline built from the same JAR/WAR you intend to run. Older baselines without method inventory need a fresh static analysis for method coverage and explicit method instrumentation.
3. Enter a session name and environment. Choose BOUNDARY for supported annotated classes/entry points; STANDARD also includes cycles and high fan-out classes; DEEP considers all included classes with safe declared method names.
4. Optionally enter comma-separated application package prefixes and exclusions. `com.example` includes that package and its descendants; it does not include `com.examples`. Exclusions win over inclusions. Prefixes affect explicit method instrumentation; framework auto-instrumentation can still emit other spans.
5. Click **Preview instrumentation plan**, then expand it. Review the selected classes/method names, omissions, version and SHA-256. Plans are bounded at 200 classes, 100 method names per class, 3,000 names overall and 24,000 encoded bytes. Changing the form invalidates the preview. Creation computes and saves the same deterministic plan for the submitted configuration.
6. Keep **Strict service + artifact declaration** selected unless you are intentionally integrating older telemetry. Strict sessions require the generated `service.name` and `archintel.artifact.sha256` resource attributes on every span batch. These are declarations made by the sender, not proof of the running binary.
7. Click **Create runtime session**. Copy the generated environment variables into the PowerShell terminal that will start your Java application. Replace the Java-agent and application-JAR paths in the example command with their actual paths. Merge the supplied artifact attribute with any resource attributes you already use. The bearer token is shown only at creation or rotation; do not share screenshots containing it.
8. Click **Start**, then launch the Java application from that terminal with the OpenTelemetry Java agent. ArchIntel uses OTLP HTTP/protobuf; it does not expose an OTLP gRPC listener.
9. Exercise representative workflows. Watch the session move from WAITING_FOR_AGENT to RUNNING and RECEIVING_DATA. A disconnected indicator means no recently accepted batch; it does not necessarily mean the application stopped.
10. Inspect **Coverage**, **Quality**, **Graph**, **Mapping**, **Flows** and **Database**. Select more than one session only when they use the same static baseline. The existing runtime assistant continues through the Phase 2 model gateway when authorized and enabled.
11. Stop workload traffic and allow the exporter to flush before clicking **Stop**. A completed report is frozen. Further telemetry is rejected; create another session for additional observations.
12. Use **Export evidence** to save the selected report. Archived individual reports remain available after raw span retention expires; expired raw sessions cannot be recombined into a new multi-session report.

Read the evidence correctly

Display	Meaning
Class coverage	Static classes identified in received spans
Method coverage	Declared method signatures matched by exact JVM descriptor, or a unique owner + method name
Entry-point coverage	Observed methods among supported explicit route/listener/scheduler annotations
Dependency coverage	Observed distinct directed class pairs among static METHOD_CALL pairs
Domain coverage	Domains with at least one observed class; this does not establish full behavior coverage
Declared matches	Accepted spans whose service/artifact declarations match the session
Unverified	Compatible-mode spans missing a service or artifact identity
Duplicate	A trace/span identity already stored in this session; it is not counted twice
Conflict	A duplicate identity carrying different sanitized data; the first observation is retained
Missing parent	Parent span has not arrived in the selected session; arriving later can repair the graph while active
Invalid parent link	A malformed cycle in parent span identities; those links are omitted
Dropped	The session's configured span-storage limit was reached
AMBIGUOUS method mapping	Multiple overload signatures fit a method name; no overload receives guessed coverage

Unobserved never means unused. Standard Java-agent method instrumentation may omit descriptors; exact overload coverage remains unavailable for those spans. Java line/branch coverage, inherited/composed route discovery and complete transaction/business-workflow coverage are not claimed.

Troubleshooting

Symptom	Action
No traces	Verify the session is started, the Java process has the generated environment, endpoint is reachable from that process, agent JAR exists, and you exercised a workflow. Inspect its exporter diagnostics.
HTTP 401	Use this session's latest bearer token. **New token** revokes the old token; update the Java process configuration and restart it as necessary.
HTTP 409 on ingestion	Check session state and service/artifact resource declarations. Any explicit mismatch is rejected even in compatible mode. Fix configuration and send future observations; the rejected batch is not stored.
HTTP 400	Check OTLP encoding, IDs, timestamps and batch limits. Malformed payloads are rejected without retaining raw bodies.
Partial success / dropped count	The default session cap is 50,000 spans. Create a new bounded observation session; a full session cannot regain capacity by retrying the same dropped spans.
Missing/ambiguous methods	Inspect method mapping diagnostics, use the matching static artifact, refresh an old method inventory, and provide descriptors where the instrumentation supports them.
FAILED after API restart	Report generation was interrupted. Retry **Stop** while raw evidence is retained. `/api/v1/runtime-sessions/{id}/diagnostics` explains recovery and retention state.

Symptom	Action
Old report has no quality panel	Historical reports are preserved without fabricating diagnostics. Create a new observation session for the new quality metadata.

The local single-process prototype does not certify every Java/framework version or production overhead. Review the explicit plan before running against your application.

Demonstrated result

The Phase 3 validation ran the pinned Spring PetClinic sample with the real Java agent in an isolated database, collected 190 spans across 83 traces, increased observed classes from 5 to 11, and retained identical reports across API restart. No existing workspace analyses or accounts were changed by that validation. See verification and machine-readable evidence.